LDC6004M – Mobile Application Development - Assessment Brief

Contents

Module Details			
Module code:	LDC6004M	**Level of Study:**	6
Module Leader(s):	Mrs.Samanthi Eranga Rubasin Siriwardana	**Credits:**	20
Assessment format:	Written feedback within Turnitin/Moodle A practical portfolio showcasing the design and development of a mobile application and a reflective narrative (**1500 - 2000 words**) that critically evaluates the development process	**Method of submission:**	Turnitin within Moodle (*submission link made available upon assignment announcement*) Portfolio (Anonymous)
Deadline or Assessment Period:	**11th June 2026, 12 noon**	**Feedback date and place:**	**1st July 2026**
Assessment limits: length, load, word count, etc.	N/A	**Component number:**	1 of 1
Is this exempt from anonymous marking under the policy?	NO	**Component weighting:**	100%

Assignment Description

You are required to design, develop, and document a mobile application on the given topic that demonstrates the core principles, techniques, and practices covered throughout the module.

The topic is **"Smart Student Planner**". Your application must include the following user requirements, along with any additional features needed to fully meet the application objectives.

1. Login / Logout
2. Dashboard (Home)
3. Task Management
    - AddTask(title, module, due_date, priority, notes)
    - EditTask(task_id)
    - DeleteTask(task_id)
    - MarkTaskComplete(task_id) / MarkTaskIncomplete(task_id)
    - SearchTasks(keyword)
4. Validation & Error Handling

The submission consists of **one assessment component**, **which comprises two integrated parts:**

- **Part I- Practical Mobile Application Portfolio**
- **Part II- Reflective Narrative**

Your work should demonstrate technical competence, design awareness, and critical reflection, aligned with industry-standard mobile development practices.

<u>Part I – Practical Mobile Application Portfolio</u>

You must design and implement a working mobile application using any one of the following frameworks:

- **Kivy (recommended)**
- **Flutter**
- **React Native**

Functional Requirements:

Your application must include:

1. Multi-screen navigation (e.g. login, home, settings, detail screens)
2. User input handling (forms, buttons, toggles)
3. Local data persistence (e.g. preferences, settings, saved content)
4. State management between screens
5. Error handling and input validation
6. Responsive UI design suitable for mobile devices

Portfolio Artefacts (Mandatory):

Submit the following as part of your portfolio:

1. Application Source Code

    - Clearly structured and well-commented
    - Stored in a version-controlled repository (e.g. GitHub)

Assignment Description

2. Design Documentation
   - App purpose and target users
   - Wireframes or UI sketches
   - Navigation flow diagram
   - Architecture overview (e.g. MVC / MVVM)

3. Screenshots
   - Demonstrating key features and user flows

4. README File
   - Installation and run instructions
   - Framework and tools used
   - Known limitations or future enhancements

Part II – Reflective Narrative

You must submit a **1,500 to 2000 word** reflective report that critically evaluates your development journey.

Your narrative should address:

Design decisions
- Choice of framework
- UI/UX and navigation decisions

Technical implementation
- State management and data persistence
- Architectural approach

Challenges and problem-solving
- Technical issues encountered
- How problems were resolve

Testing and quality assurance
- Testing strategies used

Professional practice
- Use of version control
- Coding standards and documentation

Learning reflection
- Skills developed
- What you would improve in future iterations

You are encouraged to reference module concepts, industry practices, and relevant learning resources where appropriate.

Assignment Description

Submission Requirements

You must submit the following **files as one document(one submission) which includes:**

- Practical Mobile Application Portfolio (Application source code, Design documentation and README file)
- Reflective narrative (PDF or Word)

*****A GitHub repository link must be included in your submission.**

Learning Outcomes

You must successfully achieve the following Learning Outcomes to pass this assessment:

This coursework is designed to achieve the following Programme Learning Outcomes (PLOs):

PLOs 6.1, 6.4, 6.5

6.1 Critically apply skills, techniques, and knowledge from a range of data analysis methods and algorithms for enhancing and solving problems in various domains.

6.4 Critically evaluate and analyse advanced computer science topics, and concepts, and implement them in workplace.

6.5 Identify and implement appropriate programming and software tools in AI and Generative AI to solve real word problems

Advice and Guidance

Guidelines for Students:

1. **Academic Integrity and Original Work**
   - You must submit your own original work (design, code, and written reflection).
   - Any external sources (documentation, tutorials, UI kits, code examples, libraries, blog posts, forums) must be acknowledged appropriately in your report and/or documentation.
   - Academic misconduct includes (but is not limited to): plagiarism, contract cheating, submitting another person's code, or presenting unreferenced external code as your own.
   - Similarity checking will be used (Turnitin for written work; code may also be reviewed for similarity and originality).

Using external code appropriately

It is acceptable to adapt small, clearly justified snippets (e.g., framework documentation examples), but you must:

- add clear inline comments in the code stating what was adapted and from where, and
- explain in your reflective narrative why you used/adapted it and how you integrated it.

Advice and Guidance

- Avoid copying large sections of application code or "template apps" from repositories/tutorial series. If your structure or features closely match a tutorial, this will weaken your evidence of independent development.

2. **Approach to the Assessment**

This assessment is designed to evaluate your ability to design, develop, document, and critically evaluate a mobile application using an industry-relevant workflow. Your work should demonstrate:

A. Technical competence (Part I – Portfolio)

- You must implement a working mobile app using one framework: Kivy, Flutter, or React Native.

Your application must include all required features:

- Multi-screen navigation (e.g., login/home/settings/detail screens)
- User input handling (forms, buttons, toggles)
- Local data persistence (e.g., preferences, settings, saved content)
- State management between screens
- Error handling and input validation
- Responsive UI suitable for mobile devices

You are assessed not only on whether the app runs, but also on:

- code structure, readability, and maintainability
- appropriate architecture (e.g., MVC/MVVM where relevant)
- quality of validation/error handling and robustness
- evidence of meaningful testing and debugging
- professionalism: documentation and version control

B. Critical reflection (Part II – Reflective Narrative)

Your reflective narrative (1,500–2,000 words) should critically evaluate:

- design and framework choices (with justification)
- UI/UX and navigation decisions
- implementation decisions (state management, persistence, architecture)
- challenges encountered and how you solved them
- testing strategies and quality assurance
- professional practice (Git usage, coding standards, documentation)
- what you learned and what you would improve next time

3**. Report Preparation**

Although your submission includes multiple artefacts, they should feel like one coherent portfolio, with consistent naming and clear links between evidence.

Advice and Guidance

What to include in your portfolio documentation

- App purpose and target users (what problem does it solve? who benefits?)
- Wireframes or UI sketches (early UI thinking, not just final screenshots)
- Navigation flow diagram (screens and transitions)
- Architecture overview (how your app is organised; how state and persistence are handled)
- Screenshots showing key user journeys and core features
- README with setup, run instructions, and limitations/future enhancements

Please refer the Part II – Reflective Narrative under assessment description.

Reflective Narrative: good practice

- Use headings aligned to the brief (Design decisions, technical implementation, Challenges, Testing, Professional practice, Learning reflection).
- Include short supporting evidence where helpful (e.g., small code snippets, Git screenshots, test logs) but avoid dumping large blocks of code into the narrative.
- Make your evaluation specific: explain what you chose, what worked well, what didn't, and what you would change next.

4. Code Submission Guidelines

Code must be well-structured and commented, following conventions for your chosen framework/language.

- **Kivy (Python): use docstrings, sensible file organisation, consistent naming (PEP8 where practical). (Recommended)**
- Flutter (Dart): organise into widgets/screens, separate services/state, follow common Dart style.
- React Native (JavaScript/TypeScript): component structure, separation of UI and logic, consistent formatting/linting where possible.
- Implement input validation, error handling, and responsive layout across devices/screens as required.

Version control (mandatory expectation)

- Your code must be stored in a version-controlled repository (e.g., GitHub) and your submission must include the repository link.
- Use commits to show development progress (not a single final commit).

README file (required)

- Your README must clearly explain:
- how to install dependencies and run the project
- the framework/tools used (including versions if possible)
- any environment setup steps (SDKs, emulators, build tools)
- known limitations and future enhancements

Advice and Guidance

5. **Referencing and Use of Sources**

You are not expected to write a research-heavy essay, but you must reference any external materials used to support:

- your technical decisions,
- your design reasoning (UI/UX),
- any industry practices you discuss (e.g., state management patterns, architecture approaches).

Use York St John Harvard Referencing consistently in your reflective narrative and design documentation (where appropriate).

- For code-related sources:
- cite inline in code comments for any adapted snippet, and
- list the source in your references section (or a “References” section in the narrative/documentation).

Example code comment style (adapt as appropriate):

// Adapted from official documentation: (source title, access date)

6. Word Count and Structure

Your reflective narrative must be **1,500–2,000** words.

- Keep writing focused and evidence-based:
- use headings and short paragraphs,
- use bullet points where appropriate,
- include small code snippets only when they illustrate a key point (avoid full files).
- Quality is prioritised over length, do not add filler.

7. Deadlines, Extensions, and Feedback

- Submit via Turnitin within Moodle by the published deadline.
- Late submission without an approved extension may result in penalties in line with university policy.
- Extensions may be requested according to university regulations and eligibility criteria.
- Feedback will be provided in Turnitin/Moodle as written comments (including overall feedback and/or inline notes).

Please ensure you follow the York St John University Code of Practice for Assessment and Academic Related Matters (2023–24) and pay particular attention to the academic misconduct policy.

Advice and Guidance

8. **Support and Resources**

- Use module lecture materials, lab activities, and recommended reading to guide your development and reflection.
- Study skills: If you require support with your study skills, please visit https://www.yorksj.ac.uk/students/study-skills/

Use official framework documentation for reliable implementation guidance:

- Kivy documentation
- Flutter documentation
- React Native documentation

Start early, test frequently, and use your Git commits to evidence steady progress.

Please refer to the York St John University Code of Practice for Assessment and Academic Related Matters 2023-24.
We ask that you pay particular attention to the academic misconduct policy. Penalties will be applied where a student is found guilty of academic and/or ethical misconduct, including termination of programme.(https://www.yorksj.ac.uk/media/content-assets/registry/policies/code-of-practice-for-assessment/27.Academic-misconduct-policy-202526-V2.pdf)

You are required to keep to the word limit set for an assessment and to note that you may be subject to penalty if you exceed that limit. You are required to provide an accurate word count on the cover sheet for each piece of work you submit (https://www.yorksj.ac.uk/media/content-assets/registry/policies/code-of-practice-for-assessment/26.Agreed-sanctions-policy-202526.pdf)

For late or non-submission of work by the published deadline or an approved extended deadline, a mark of NS will be recorded. Where a re-assessment opportunity exists, a student will normally be permitted only one attempt to be re-assessed for a capped mark (https://www.yorksj.ac.uk/media/content-assets/registry/policies/full-code-of-practice-for-assessment/Code-of-Practice-for-Assessment-2024-25-V2.pdf)

An extension to the published deadline may be granted to an individual student if they meet the eligibility criteria of the (https://www.yorksj.ac.uk/media/content-assets/registry/policies/code-of-practice-for-assessment/13.Exceptional-circumstances-policy-202526.pdf)

How is this assessment marked?

Your work will be assessed according to the assessment instructions provided in this document and the selected Learning Outcomes (LOs) for this module. You are expected to

How is this assessment marked?

demonstrate achievement of the stated LOs through clear, structured, and academically appropriate responses aligned with the task requirements.

This assessment uses a Fixed Scale Marking approach aligned with the University's Generic Assessment Descriptors. For each assessment criterion, markers will select the most appropriate descriptor level (e.g., Third, 2:2, 2:1, First, High First, Borderline Fail, or Fail) and assign the corresponding fixed scale mark. Marks are not awarded flexibly within percentage bands; instead, a specific fixed mark is selected based on how accurately and consistently your work meets the descriptor standard.

This approach ensures consistency, transparency, and comparability across all assessments, regardless of discipline or mode of submission. It also ensures that assessment decisions are aligned with institutional quality standards.

To achieve a pass, you must meet the minimum threshold of 40%, demonstrating achievement of the required Learning Outcomes and baseline assessment expectations. In addition, to achieve a higher classification (e.g., 2:2, 2:1, First, High First), your work must meet the full set of standards described at that level. This means demonstrating progressive improvement in areas such as knowledge and understanding, critical analysis, application of theory, structure and organisation, academic writing, referencing, and overall quality of argument

Marking Criteria – Mobile Application Development		
Practical Mobile Application Portfolio		
Question	Criteria	
Mobile Application Concept, Design Documentation, and UI Implementation	Learners should demonstrate effective UI design and navigation for mobile devices. Also, learners should demonstrate clear design planning and documentation	
	• Clear explanation of app purpose, problem domain, and target users	
	• Wireframes / UI sketches showing screen layout and navigation	
	• Navigation flow diagram demonstrating multi-screen structure	
	• Architecture overview (e.g. MVC / MVVM) with justification	
	• Implementation of multi-screen navigation (e.g. login, home, settings)	
	• Responsive UI suitable for different mobile screen sizes	
	• Appropriate use of UI components (forms, buttons, inputs)	
User Input Management, Validation, and	Learners should demonstrate correct handling of user input and validation. Also, Learners should demonstrate effective management of application state across screens.	

Application State Control	• Correct handling of user inputs (forms, buttons, toggles)	
	• Input validation to prevent invalid or unsafe input	
	• Error handling with meaningful user feedback	
	• Clear explanation of state management approach	
	• Correct implementation of state sharing between screens	
	• Evidence of maintaining consistency and avoiding data loss	
Local Data Persistence	Learners should demonstrate correct data persistence techniques.	
	• Explanation of chosen persistence method (e.g. local storage/JSON/db)	
	• Correct implementation of save + load behaviour	
	• Error handling and data integrity considerations	
Code Quality, Version Control & README	Learners should demonstrate professional development practices.	
	• Well-structured, readable, and appropriately commented code	
	• Effective use of Git/version control (repo structure, meaningful commits)	
	• README: install/run steps, tools/framework, known limits/future work	
Functional Requirements Coverage & Working App Evidence	Learners should provide complete and appropriate portfolio evidence	
	• Evidence of **all required features** implemented (navigation, input, persistence, state, validation/error handling, responsive UI)	
	• Screenshots demonstrating key user flows and core features	
	Part II – Reflective Narrative	
Professional Design, Technical Decision-Making, Problem Solving, and Academic Reflection	Learners should critically reflect on design and implementation decisions • Reflection on design choices (UI/UX, navigation, framework selection) • Discussion of architectural and technical decisions Learners should demonstrate critical evaluation of development challenges • Identification of technical challenges encountered • Explanation of problem-solving strategies and resolutions • Discussion of testing strategies and quality assurance Learners should reflect on professional skills and future development. • Reflection on use of version control, documentation, and coding standards • Skills developed and learning gained during the project • Suggestions for future improvements or extensions Learners should demonstrate professional report presentation • Correct referencing using an appropriate citation style • Clear, structured, and coherent reflective writing • Professional presentation and formatting	

Detailed Marking Criteria

Pass Grade Bands (100 – 40) (Learning Outcomes must be met)
Fail Grade Bands (39 – 0) (Learning Outcomes are not met)
Note that the detailed marking criteria below are derived from the YSJU GAD table

General Characteristics	3rd (40, 42, 45, 48)	2:2 (52, 55, 58)	2:1 (62, 65, 68)	First (72, 75, 78, 82)	High First (85, 88, 92, 95, 98, 100)	Borderline Fail (32,35, 38) **(Credits may be compensated)**	Fail (0, 2, 5, 8, 12, 15, 18, 22, 25, 28) **(Credits may not be compensated)**
Adjectives	**Satisfactory**	**Good**	**Very Good**	**Excellent**	**Exemplary**	**Not Successful**	**Fail**
Mobile Application Concept, Design Documentation, and UI Implementation	Satisfactory understanding demonstrated. App purpose, problem domain, and users are clearly identified but lack depth. Wireframes/UI sketches show basic screen layouts and navigation structure. Navigation flow diagram represents multi-screen structure but may lack refinement. Architecture overview (e.g., MVC/MVVM) is described with limited critical justification. Multi-screen navigation is	Good standard of work with clear structure. App purpose and problem domain are well explained with appropriate target user identification. Wireframes/UI sketches are clear and logically structured. Navigation flow diagram accurately demonstrates multi-screen relationships. Architecture overview is appropriate with some justification of design choice. Multi-screen navigation is correctly	Very good analytical and practical standard. App purpose, problem domain, and users are clearly defined with strong contextual understanding. Wireframes/UI sketches are detailed, labelled, and professionally structured. Navigation flow diagram is clear, accurate, and logically organised. Architecture choice (MVC/MVVM) is clearly justified with understanding of separation of concerns. Multi-screen navigation is	Excellent and well-developed submission. App purpose and problem domain are critically articulated with clear user-centred focus. Wireframes/UI sketches are detailed, professional, and demonstrate strong design thinking. Navigation flow diagram is comprehensive and clearly represents application logic. Architecture overview is well-justified, demonstrating critical understanding of design patterns.	Exceptional and professional-standard documentation and implementation. App purpose, problem domain, and target users are defined with outstanding clarity and industry awareness. Wireframes/UI sketches are highly detailed, well-labelled, and demonstrate professional UX design principles. Navigation flow diagram is comprehensive, precise, and clearly models a scalable multi-screen architecture.	Basic attempt made but significant weaknesses remain. App purpose and target users are vaguely explained. Wireframes/UI sketches are simplistic or incomplete. Navigation flow diagram lacks clarity or accuracy. Architecture overview is descriptive but lacks justification. Multi-screen navigation is partially implemented with errors. UI shows limited responsiveness and inconsistent layout. Basic UI components are used but not	Work does not meet Level 6 standards. App purpose, problem domain, and target users are unclear or missing. Wireframes/UI sketches are absent or extremely limited. Navigation flow diagram is missing or incorrect. Architecture overview is absent or fundamentally misunderstood. Multi-screen navigation is not implemented or not functional. UI is poorly structured and not suitable for mobile devices.

	implemented and functional. UI is generally appropriate for mobile devices with minor layout inconsistencies. Appropriate UI components are used correctly.	implemented and mostly smooth. UI demonstrates good responsiveness across screen sizes. Effective and appropriate use of forms, buttons, and inputs.	smoothly implemented and logically structured. UI is responsive, consistent, and suitable for different mobile screen sizes. UI components are well-selected and effectively implemented.	Multi-screen navigation is robust and seamlessly integrated. UI design shows high consistency, responsiveness, and usability awareness. Advanced and appropriate use of UI components with attention to user experience	Architecture overview (MVC/MVVM) demonstrates critical evaluation and strong justification aligned with industry best practice. Multi-screen navigation is fully functional, scalable, and elegantly structured. UI is highly responsive, consistent across screen sizes, and demonstrates strong usability and accessibility awareness. UI components are used strategically and professionally to enhance user experience.	appropriately structured.	Very limited use of appropriate UI components.
User Input Management, Validation, and Application State Control	Satisfactory implementation meeting minimum Level 6 standards. User inputs are handled correctly in most cases. Basic validation prevents common invalid inputs. Error handling provides	Good and consistent implementation. User inputs (forms, buttons, toggles) handled correctly and reliably. Validation covers common edge cases and prevents unsafe input. Error messages are clear and helpful to users.	Very good technical and conceptual understanding. User inputs are handled efficiently with clean logic. Validation is robust and anticipates edge cases. Error handling is user-friendly and enhances usability.	Excellent implementation demonstrating strong problem-solving ability. User input handling is comprehensive and professionally structured. Validation logic is thorough, secure, and well-designed.	Exceptional and industry-standard implementation. User input handling is precise, scalable, and professionally engineered. Validation includes advanced handling of edge cases and demonstrates awareness of	Basic attempt but significant weaknesses. Some user inputs handled but inconsistently. Minimal validation with limited protection against invalid input. Basic or unclear error messages.	Does not meet Level 6 expectations. User inputs are not handled correctly or application crashes occur. No meaningful input validation implemented. Error handling is absent or misleading. State management approach is missing

	understandable feedback. State management approach is described clearly but lacks depth. State sharing between screens works in standard scenarios. Minor inconsistencies may occur but no major data loss.	State management approach is clearly explained and logically structured. State sharing between screens is correctly implemented. Application maintains data consistency across navigation with minimal issues.	State management approach (e.g., shared model, controller logic) is clearly justified. State sharing between screens is seamless and reliable. Strong evidence of maintaining consistency and preventing data loss across user sessions.	Error handling enhances user experience with meaningful and contextual feedback. State management approach is clearly articulated with critical understanding. State sharing across screens is robust and well-architected. Application demonstrates consistent behaviour under varied user interactions with no data integrity issues.	security considerations. Error feedback is highly intuitive and improves overall UX quality. State management approach is critically evaluated and aligned with best practice architecture (e.g., MVC/MVVM separation). State sharing is seamless, maintainable, and extensible. Application demonstrates full data integrity, consistency, and resilience across all screens and interactions.	State management approach is described superficially. Partial state sharing between screens with inconsistencies. Some data inconsistency or loss during navigation.	or fundamentally incorrect. State is not shared correctly between screens. Frequent data loss or inconsistent behaviour across navigation.
Local Data Persistence	Minimum acceptable standard. Basic persistence is implemented. Explanation of the chosen method is brief or unclear. Save and load functionality is limited and may not cover all cases. Little consideration is given to error handling or data integrity.	Adequate implementation of local data persistence. Persistence method is identified but explanation lacks clarity or justification. Save and load behaviour works but may be inconsistent. Error handling is minimal and data integrity considerations are basic.	Good understanding of data persistence concepts. Persistence method is explained clearly but with limited depth. Save and load behaviour functions correctly in most cases. Basic error handling is present, though data integrity checks may be limited.	Very strong implementation of data persistence. Clear explanation of the chosen method is provided. Save and load functionality works correctly with only minor limitations. Error handling and data integrity are well considered, with small improvements possible.	Outstanding demonstration of local data persistence aligned with industry best practice. The chosen persistence method (e.g. local storage, JSON, database) is clearly explained and well justified. Save and load behaviour is correctly and reliably implemented in all	Limited evidence of correct data persistence. Explanation of the persistence method is weak or incorrect. Save and load behaviour is unreliable or partially implemented. Error handling is largely missing, risking data loss. Credits may be compensated.	Little or no evidence of data persistence. No clear explanation of persistence method. Save and load behaviour is missing or non-functional. No consideration of error handling or data integrity. Learning outcomes have not been met. Credits may not be compensated.

					scenarios. Robust error handling is present, ensuring strong data integrity and preventing data loss or corruption.		
Code Quality Version Control & README,	Minimum acceptable standard. Code functions but has poor structure or limited commenting. Git/version control usage is minimal, with few commits or unclear messages. The README is brief and incomplete, providing limited guidance on setup or usage.	Adequate code quality. Code is generally readable but may be inconsistently structured or commented. Git/version control is used at a basic level with limited or generic commits. The README includes basic installation or runs instructions but lacks detail or clarity.	Code is well structured and readable, with sufficient commenting. Git/version control is used correctly, though commit messages or structure may lack consistency. The README provides clear installation and usage instructions but offers limited discussion of limitations or future work.	Very strong code quality with clear structure, good readability, and appropriate commenting. Git/version control is used well with meaningful commits and a logical repository structure. The README clearly explains how to install and run the application and describes tools/frameworks used, with some discussion of limitations or future work.	Code is exceptionally well structured, highly readable, and consistently commented following professional standards. Git/version control is used effectively with a clear repository structure, frequent and meaningful commits, and appropriate commit messages. The README is comprehensive, clearly documenting installation and run steps, tools/frameworks used, known limitations, and realistic future improvements.	Limited evidence of professional development practice. Code is difficult to read or poorly structured. Git/version control usage is weak or inconsistent. The README is incomplete or unclear, missing key information. Credits may be compensated.	Little or no evidence of professional practice. Code is poorly structured or unreadable. Git/version control is not used appropriately or not used at all. No meaningful README is provided. Learning outcomes have not been met. Credits may not be compensated.
Functional Requirements Coverage & Working App Evidence	Minimum acceptable standard. Some required features are implemented, but others are	Adequate coverage of functional requirements. Core features are implemented, but some required	Good evidence of functional implementation. Most required features are implemented	Very strong evidence of functional coverage. All major required features are implemented and	Comprehensive and professional evidence demonstrating that all required functional features	Limited evidence of functional coverage. Several required features are missing or not working correctly.	Little or no evidence of functional requirements being met. Required features are largely missing or non-

	incomplete or weak. Screenshots provide limited evidence of functionality. Application works at a basic level but lacks completeness or robustness.	functionality may be limited or partially implemented. Screenshots demonstrate basic user flows, though evidence may lack clarity or completeness.	correctly, with minor limitations. Screenshots demonstrate main user flows, though some features may not be fully evidenced or clearly presented. The application is largely functional.	working as expected. Screenshots clearly illustrate key user flows and core functionality. Minor gaps in evidence or presentation may exist but do not affect overall app functionality.	are fully implemented, including navigation, user input handling, data persistence, state management, validation/error handling, and responsive UI. Screenshots clearly demonstrate complete user flows and core features. Evidence strongly confirms a fully working and reliable application.	Screenshots are incomplete, unclear, or poorly labelled. Evidence does not fully demonstrate a working application. Credits may be compensated.	functional. Screenshots are absent or do not demonstrate app functionality. The application does not meet learning outcomes. Credits may not be compensated.
Professional Design, Technical Decision-Making, Problem Solving, and Academic Reflection	Satisfactory reflective practice demonstrated. Design and framework choices are explained with some justification. Architectural and technical decisions are discussed but lack critical depth. Key development challenges are identified with basic explanation of solutions. Testing strategies are mentioned. Some reflection on version control, documentation, and skills gained.	Good critical reflection with clear structure. Design decisions (UI/UX, navigation, framework) are explained and justified. Architectural choices are discussed with logical reasoning. Technical challenges are clearly identified and problem-solving strategies explained. Testing strategies and quality assurance approaches are described appropriately.	Very good analytical and reflective depth. Design and implementation decisions are critically evaluated rather than described. Clear discussion of architectural trade-offs and technical reasoning. Development challenges are analysed with thoughtful problem-solving reflection. Testing strategies are clearly explained and linked to quality assurance.	Excellent critical reflection demonstrating maturity of thought. Design choices are evaluated using reasoned argument and awareness of alternatives. Architectural and technical decisions are critically justified with clear understanding of best practice. Technical challenges are deeply analysed with strong evidence of systematic problem-solving. Testing strategies are thorough and	Exceptional and industry-standard critical reflection. Design, UI/UX, navigation, and framework choices are critically evaluated with sophisticated insight and awareness of alternative approaches. Architectural and technical decisions demonstrate advanced understanding of design principles and industry standards. Development challenges are analysed in depth,	Limited reflective depth. Design choices and architectural decisions are described superficially without analysis. Technical challenges are mentioned but not critically evaluated. Minimal explanation of problem-solving strategies. Limited discussion of testing approaches. Professional skills reflection is brief and lacks insight. Referencing and structure show weaknesses;	Work does not meet Level 6 expectations. Little or no reflection provided. Design and technical decisions are merely described or missing. No meaningful discussion of challenges or problem-solving strategies. Testing and quality assurance not addressed. No reflection on professional skills development. Report lacks structure, referencing, and academic

	Future improvements identified but limited in scope. Report is structured and readable with mostly correct referencing.	Reflects on professional skills such as version control and coding standards. Realistic suggestions for future improvements. Report presentation is clear, coherent, and correctly referenced.	Strong reflection on professional development, including use of Git/version control, documentation practices, and coding standards. Insightful evaluation of skills gained and areas for growth. Well-structured academic writing with accurate referencing and professional formatting.	clearly linked to application quality and reliability. Professional reflection shows strong awareness of industry practices (version control workflows, documentation, standards). Clear articulation of transferable skills and realistic future development pathways. Report is professionally presented, logically structured, and consistently referenced.	demonstrating strategic and innovative problem-solving. Testing and quality assurance approaches are systematic, rigorous, and professionally articulated. Professional skills reflection demonstrates advanced awareness of software engineering practices, including structured version control workflows, documentation discipline, and coding standards. Future improvements are strategically proposed, realistic, and technically sound. Report demonstrates exemplary academic writing, flawless structure, consistent referencing, and professional formatting throughout.	presentation inconsistent.	presentation standards.